

Relatório do Projeto

Jogo Tic Tac Toe

Unidade Curricular: Programação Orientada a Objetos

Curso: Engenharia Informática

Ano letivo: 2025/2026

Grupo: PL7

Elementos do grupo:

- Artem Chernousov — 202300991
 - Josebe Paulo Silva — 2025100504
-

Índice

1. Apresentação do Projeto
 2. Modelação do Domínio
 3. Diagrama de Classes UML
 4. Arquitetura da Aplicação
 5. Funcionalidades Implementadas
 6. Aplicação dos Conceitos de Programação Orientada a Objetos
 7. Tratamento de Exceções
 8. Testes Unitários
 9. Dificuldades Encontradas
 10. Conclusão
-

1. Apresentação do Projeto

O projeto desenvolvido consiste numa aplicação desktop em Java, com interface gráfica construída em JavaFX, baseada no jogo clássico **Tic Tac Toe**, também conhecido como jogo do galo.

O objetivo do jogo é permitir que dois jogadores humanos joguem entre si num tabuleiro de dimensão 3x3. Um dos jogadores utiliza o símbolo **X** e o outro utiliza o símbolo **O**. Os jogadores jogam alternadamente, tentando formar uma sequência de três símbolos iguais numa linha, numa coluna ou numa diagonal.

O jogador com o símbolo **X** inicia sempre a partida. Em cada jogada, o jogador atual escolhe uma célula livre do tabuleiro. Se a célula estiver disponível, o símbolo do jogador é colocado nessa posição. Caso a célula já esteja ocupada, a jogada não é permitida.

As principais regras do jogo são:

- O jogo é disputado por dois jogadores humanos.
- O tabuleiro tem 3 linhas e 3 colunas.
- O jogador **X** joga primeiro.
- Os jogadores alternam turnos após cada jogada válida.
- Vence o jogador que alinhar três símbolos iguais numa linha, coluna ou diagonal.
- Se todas as células forem preenchidas sem existir vencedor, o jogo termina em empate.
- Não é permitido jogar numa célula já ocupada.
- Não é permitido iniciar uma partida sem preencher corretamente os nomes dos jogadores.

A interação com o utilizador é feita através de uma interface gráfica em JavaFX. A aplicação apresenta um menu inicial, um ecrã de configuração dos jogadores, o tabuleiro interativo do jogo e um ecrã final com o resultado da partida.

2. Modelação do Domínio

A modelação do domínio foi realizada com base nas entidades necessárias para representar a lógica do jogo Tic Tac Toe de forma orientada a objetos.

As principais entidades identificadas foram `Game`, `Board`, `Cell`, `Player`, `HumanPlayer`, `Symbol`, `GameState` e `InvalidMoveException`.

A classe `Game` representa uma partida. Esta classe controla o tabuleiro, os jogadores, o jogador atual, o estado da partida e o vencedor. É a classe principal da lógica do jogo, sendo responsável por iniciar uma nova partida, processar jogadas, alternar o jogador atual e verificar se o jogo terminou.

A classe `Board` representa o tabuleiro do jogo. O tabuleiro é composto por uma matriz 3x3 de objetos do tipo `Cell`. Esta classe é responsável por validar posições, marcar células, verificar se uma célula está livre, verificar se o tabuleiro está cheio e verificar se existe uma condição de vitória.

A classe `Cell` representa uma célula individual do tabuleiro. Cada célula possui uma linha, uma coluna e um símbolo associado. Inicialmente, todas as células têm o símbolo `EMPTY`. Durante o jogo, uma célula pode passar a conter o símbolo `X` ou `O`.

A classe abstrata `Player` representa um jogador genérico. Esta classe contém os atributos comuns a qualquer jogador, nomeadamente o nome e o símbolo associado. A classe `HumanPlayer` herda de `Player` e representa concretamente um jogador humano.

O enum `Symbol` representa os símbolos possíveis no jogo: X, O e EMPTY. O enum `GameState` representa os possíveis estados da partida: NOT_STARTED, RUNNING, WIN e DRAW.

A classe `InvalidMoveException` representa uma exceção personalizada utilizada para assinalar jogadas inválidas, como tentar jogar fora do tabuleiro, jogar numa célula ocupada ou executar uma jogada quando o jogo ainda não está em execução.

As relações principais entre as entidades são:

- `Game` contém um objeto `Board`.
- `Game` contém dois jogadores: `playerX` e `playerO`.
- `Game` mantém uma referência para o jogador atual e para o vencedor.
- `Board` contém uma matriz de objetos `Cell`.
- `Cell` contém um valor do tipo `Symbol`.
- `HumanPlayer` herda de `Player`.
- `Game` utiliza `GameState` para representar o estado atual da partida.
- `Board` e `Game` utilizam `InvalidMoveException` para tratar jogadas inválidas.

Esta modelação permite separar a lógica principal do jogo da interface gráfica, tornando o código mais organizado, modular e fácil de testar.

3. Diagrama de Classes UML

O diagrama de classes UML representa a estrutura principal da aplicação, incluindo as classes do modelo, os respetivos atributos, métodos e relações.

As classes principais representadas no diagrama são:

- `Game`
- `Board`
- `Cell`
- `Player`
- `HumanPlayer`
- `Symbol`
- `GameState`
- `InvalidMoveException`

A relação de herança mais importante ocorre entre Player e HumanPlayer. A classe Player é abstrata e define os atributos e comportamentos comuns a qualquer jogador, enquanto HumanPlayer representa uma implementação concreta de jogador humano.

Também existem relações de composição e associação relevantes. A classe Game possui um Board e dois jogadores. A classe Board possui várias células, organizadas numa matriz 3x3. Cada Cell possui um Symbol.

O diagrama UML ajuda a compreender a organização do modelo de domínio e mostra como as responsabilidades foram distribuídas entre as classes.

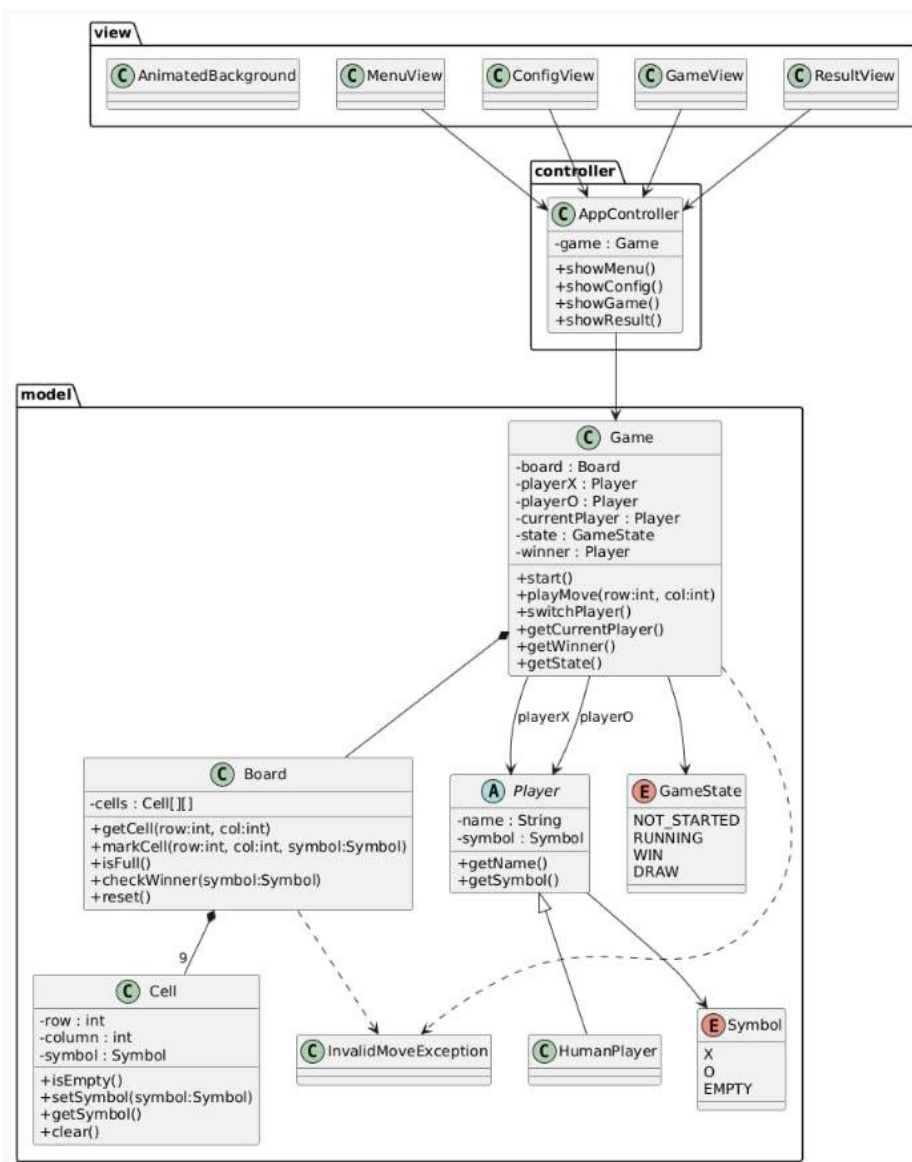


Figura 1 — Diagrama de classes UML da aplicação.

4. Arquitetura da Aplicação

A aplicação foi organizada seguindo uma separação clara entre o modelo de domínio, o controlo da aplicação e a interface gráfica. Esta organização facilita a manutenção do projeto e permite que a lógica do jogo seja testada independentemente da interface gráfica.

A estrutura principal do projeto está organizada nos seguintes packages:

```
tictactoe
├── controller
├── model
└── view
```

4.1. Modelo de Domínio

O package `model` contém as classes responsáveis pela lógica do jogo. Neste package encontram-se as classes `Game`, `Board`, `Cell`, `Player`, `HumanPlayer`, `InvalidMoveException` e os enums `Symbol` e `GameState`.

Estas classes representam o domínio do problema e não dependem diretamente da interface gráfica. Por exemplo, a classe `Board` sabe verificar se existe um vencedor, mas não sabe como esse resultado será apresentado visualmente ao utilizador.

Esta separação é importante porque permite reutilizar e testar a lógica do jogo sem depender dos componentes JavaFX.

4.2. Controlo da Aplicação

O package `controller` contém a classe `AppController`. Esta classe é responsável por coordenar a aplicação e controlar a navegação entre as diferentes vistas.

O controlador faz a ligação entre o modelo e a interface gráfica. Por exemplo, quando o utilizador introduz os nomes dos jogadores e inicia a partida, o controlador cria o objeto `Game` com os respetivos jogadores e apresenta a vista do jogo.

Também é o controlador que permite mudar entre o menu principal, a configuração, o tabuleiro e o ecrã de resultado.

4.3. Interface Gráfica

O package `view` contém as classes responsáveis pela interface gráfica desenvolvida em JavaFX. As principais classes deste package são:

- `MenuView`
- `ConfigView`
- `GameView`

- ResultView
- AnimatedBackground

A classe MenuView representa o menu principal da aplicação. A classe ConfigView permite ao utilizador introduzir os nomes dos jogadores. A classe GameView apresenta o tabuleiro interativo e permite realizar jogadas. A classe ResultView apresenta o resultado final da partida. A classe AnimatedBackground é responsável pelo fundo visual da aplicação.

A interface gráfica foi construída programaticamente em JavaFX, sem utilização de FXML. Foram usados elementos como StackPane, BorderPane, VBox, HBox, Button, Label, TextField e efeitos visuais como DropShadow.

A aplicação utiliza ainda recursos externos, como imagens e sons, organizados dentro da pasta src/main/resources.

5. Funcionalidades Implementadas

A aplicação implementa as principais funcionalidades necessárias para jogar uma partida completa de Tic Tac Toe.

5.1. Menu principal

Ao iniciar a aplicação, o utilizador visualiza um menu inicial com o logótipo do jogo, o título da aplicação e um botão para iniciar o jogo.

Esta vista serve como ponto de entrada da aplicação e permite avançar para a configuração dos jogadores.

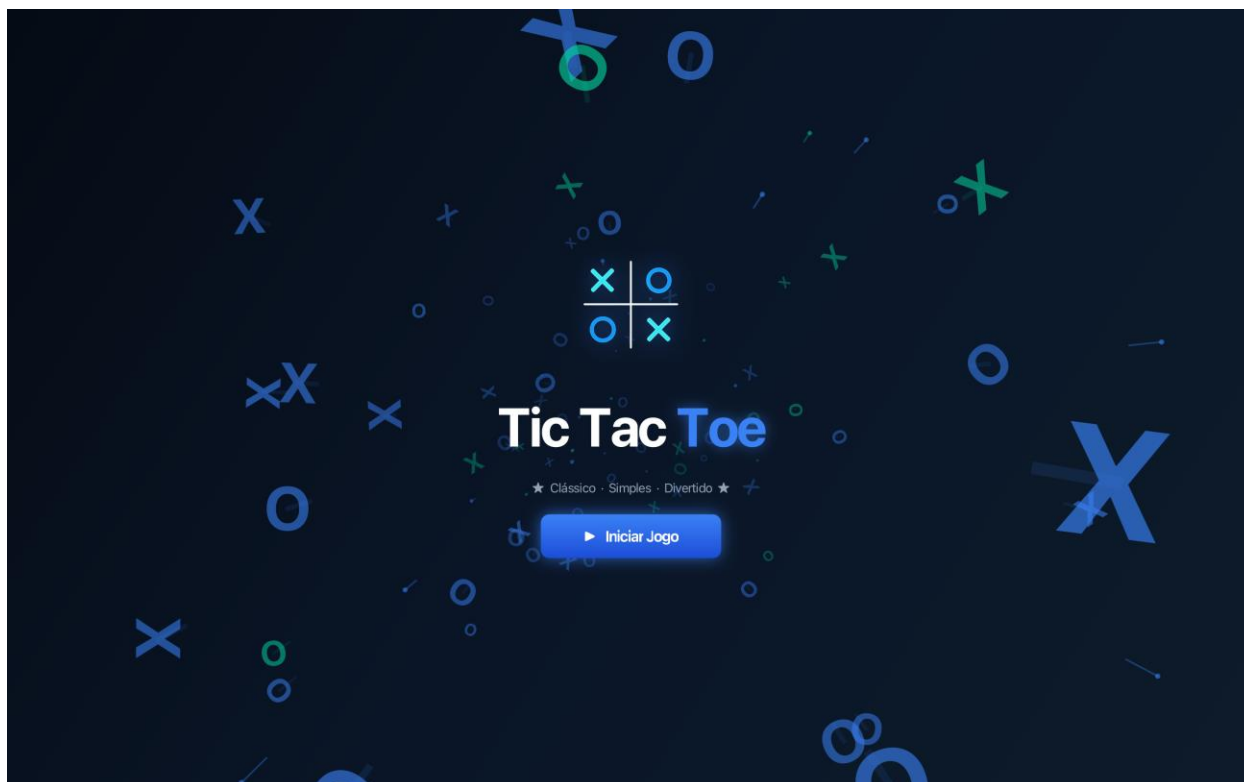


Figura 2 — Menu principal.

5.2. Configuração dos jogadores

Antes de começar a partida, o utilizador deve inserir os nomes dos dois jogadores. A aplicação valida os dados introduzidos, impedindo que o jogo seja iniciado se algum dos campos estiver vazio ou se os dois jogadores tiverem o mesmo nome.

Quando ocorre um erro de validação, é apresentada uma mensagem visual ao utilizador e os campos inválidos são destacados.

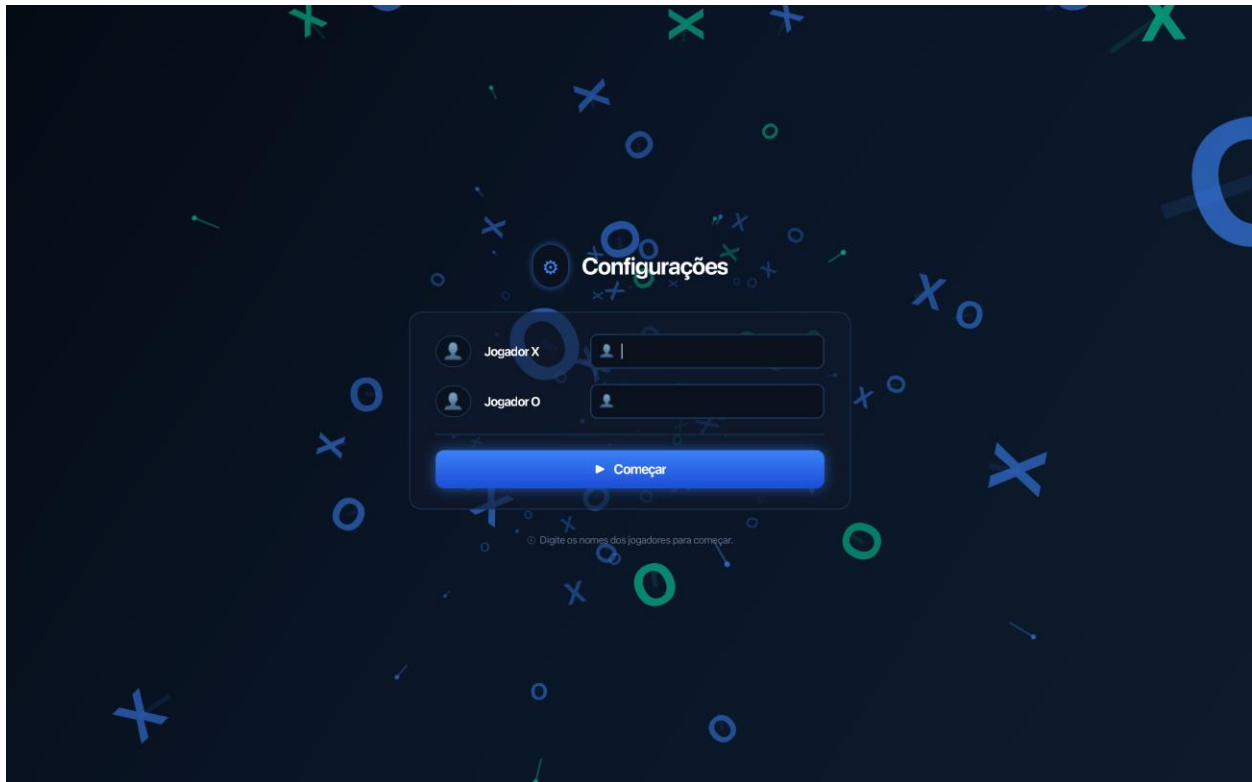


Figura 3 — Configuração dos jogadores.

5.3. Início de uma nova partida

Depois de inserir nomes válidos, é criada uma nova instância de Game. A partida começa no estado RUNNING, o tabuleiro é inicializado vazio e o jogador com o símbolo X é definido como primeiro jogador.

5.4. Interação com o tabuleiro

Durante a partida, o utilizador interage com o jogo clicando nas células do tabuleiro. Quando uma célula livre é selecionada, o símbolo do jogador atual é colocado nessa posição.

Depois de cada jogada válida, a aplicação atualiza visualmente o tabuleiro e alterna o turno para o outro jogador, caso ainda não exista vencedor ou empate.

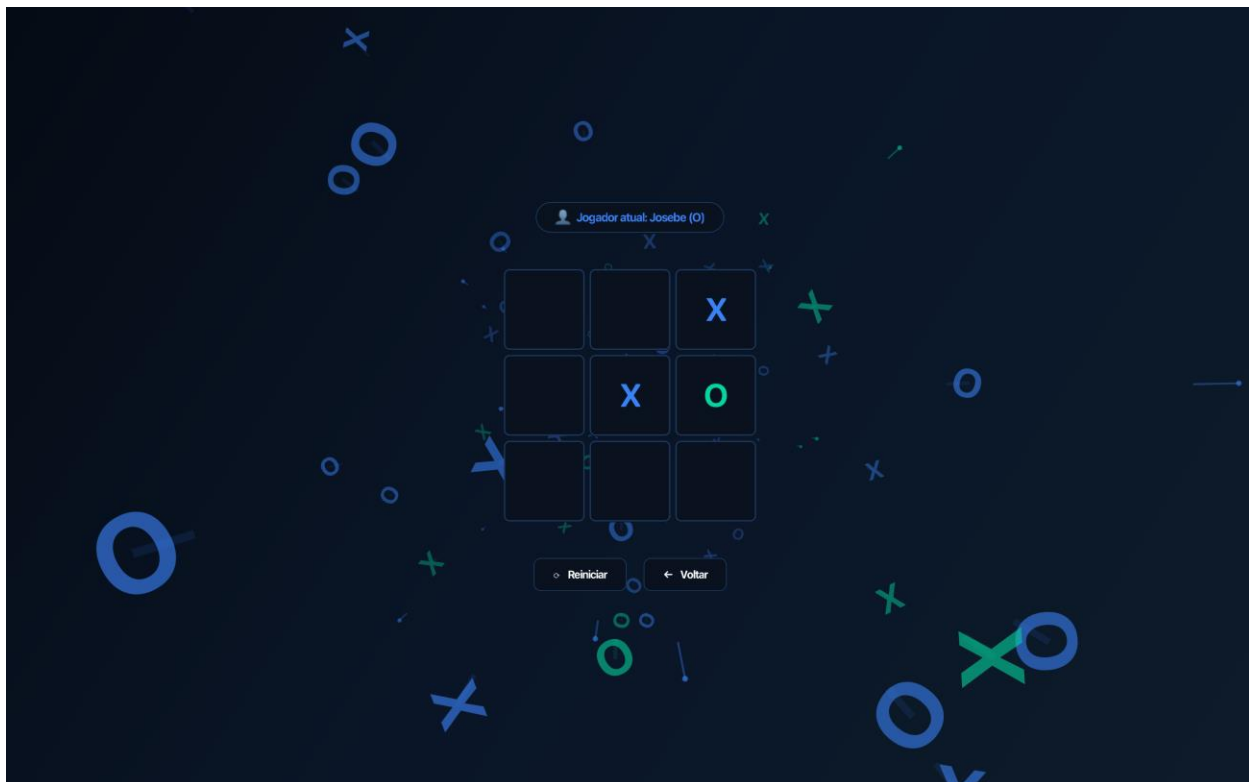


Figura 4 — Tabuleiro durante uma partida.

5.5. Validação de jogadas

A aplicação valida as ações do utilizador para garantir o correto funcionamento do jogo. Não é possível jogar numa célula ocupada, utilizar símbolos inválidos ou executar jogadas quando o jogo não está em execução.

Esta validação é feita principalmente no modelo, através das classes Board e Game.

5.6. Verificação de vitória

Após cada jogada, a aplicação verifica se o jogador atual venceu a partida. A verificação considera três possibilidades:

- três símbolos iguais numa linha;
- três símbolos iguais numa coluna;
- três símbolos iguais numa diagonal.

Se uma destas condições for satisfeita, o estado do jogo passa para WIN e o vencedor é guardado.

5.7. Verificação de empate

Se o tabuleiro ficar completamente preenchido e nenhum jogador vencer, o estado do jogo passa para DRAW. Neste caso, não existe vencedor.

5.8. Ecrã de resultado

Quando a partida termina, a aplicação apresenta um ecrã de resultado. Este ecrã indica se houve vencedor ou se a partida terminou em empate.

A partir desta vista, o utilizador pode voltar ao menu principal ou iniciar uma nova partida.

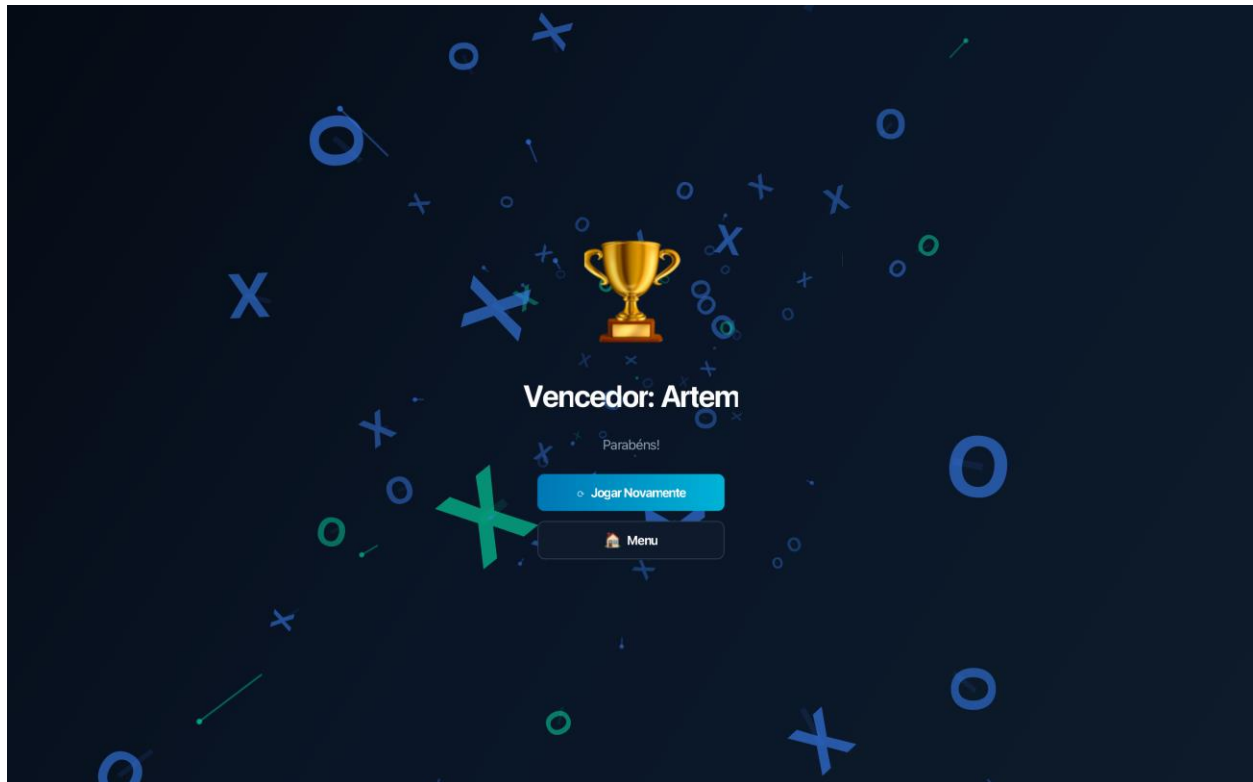


Figura 5 — Ecrã de resultado.

6. Aplicação dos Conceitos de Programação Orientada a Objetos

Durante o desenvolvimento do projeto foram aplicados vários conceitos fundamentais de Programação Orientada a Objetos, nomeadamente encapsulamento, herança e polimorfismo.

6.1. Encapsulamento

O encapsulamento foi aplicado através da definição dos atributos das classes como privados, permitindo o acesso ao estado interno apenas através de métodos públicos controlados.

Na classe `Cell`, os atributos `row`, `column` e `symbol` são privados. A leitura destes valores é feita através de métodos como `getRow()`, `getColumn()` e `getSymbol()`. A alteração do símbolo é feita através do método `setSymbol(Symbol symbol)`, que valida se o símbolo recebido não é nulo.

Na classe `Player`, os atributos `name` e `symbol` também são privados. Estes valores são definidos através dos métodos `setName()` e `setSymbol()`, que validam os dados antes de os guardar. Por exemplo, não é permitido criar um jogador com nome vazio ou com símbolo inválido.

Na classe `Board`, a matriz de células está encapsulada no atributo privado `cells`. O acesso às células é feito através do método `getCell(int row, int column)`, que valida primeiro se a posição indicada pertence ao tabuleiro.

Este uso do encapsulamento garante que os objetos mantêm um estado válido e impede alterações diretas e incorretas a partir de outras classes.

6.2. Herança

A herança foi aplicada através da relação entre as classes `Player` e `HumanPlayer`.

A classe `Player` é uma classe abstrata que define as características comuns a qualquer tipo de jogador, nomeadamente o nome e o símbolo. A classe `HumanPlayer` herda de `Player` e representa um jogador humano concreto.

Esta hierarquia permite cumprir o requisito de existência de uma estrutura hierárquica no domínio da aplicação. Além disso, deixa o projeto preparado para possíveis extensões futuras, como a criação de um `ComputerPlayer` para representar um jogador controlado pelo computador.

6.3. Polimorfismo

O polimorfismo é aplicado através da utilização do tipo abstrato `Player` na classe `Game`.

Embora os jogadores criados sejam objetos da classe `HumanPlayer`, a classe `Game` trabalha com referências do tipo `Player`, como `playerX`, `playerO`, `currentPlayer` e `winner`.

Isto significa que a lógica do jogo depende do tipo geral `Player` e não diretamente da classe concreta `HumanPlayer`. Desta forma, se futuramente for criada outra classe que herde de `Player`, como `ComputerPlayer`, a lógica principal do jogo poderá continuar a funcionar com poucas alterações.

Assim, o uso de polimorfismo contribui para tornar o projeto mais flexível, extensível e alinhado com os princípios da Programação Orientada a Objetos.

7. Tratamento de Exceções

O projeto inclui tratamento explícito de exceções para controlar situações inválidas e evitar comportamentos incorretos durante a execução da aplicação.

A principal exceção personalizada criada foi `InvalidMoveException`. Esta exceção herda de `Exception` e é utilizada para representar erros associados a jogadas inválidas.

7.1. Posições inválidas no tabuleiro

Na classe `Board`, o método `validatePosition(int row, int column)` verifica se a linha e a coluna indicadas pertencem ao tabuleiro. Como o tabuleiro tem dimensão 3x3, apenas são válidas posições entre 0 e 2.

Se o utilizador tentar aceder a uma posição fora destes limites, é lançada uma `InvalidMoveException` com uma mensagem de erro adequada.

7.2. Células ocupadas

O método `markCell(int row, int column, Symbol symbol)` verifica se a célula indicada está vazia antes de a marcar. Se a célula já estiver ocupada, é lançada uma `InvalidMoveException`.

Esta validação impede que um jogador substitua o símbolo de outro jogador.

7.3. Símbolos inválidos

Também no método `markCell`, é verificado se o símbolo recebido é válido. Não é permitido marcar uma célula com `null` ou com `Symbol.EMPTY`. Caso isso aconteça, é lançada uma `InvalidMoveException`.

7.4. Jogadas antes do início do jogo

Na classe `Game`, o método `playMove(int row, int column)` verifica se o estado atual do jogo é `RUNNING`. Se o jogo ainda estiver em `NOT_STARTED`, ou se já tiver terminado, a jogada não é aceite.

Esta verificação impede que sejam feitas jogadas fora do momento correto da partida.

7.5. Dados inválidos dos jogadores

Além da exceção personalizada, foram usadas exceções do tipo `IllegalArgumentException` para validar dados inválidos na criação de jogadores e partidas.

Na classe `Player`, não é permitido criar jogadores com nome vazio, nome nulo, símbolo nulo ou símbolo `EMPTY`.

Na classe Game, não é permitido criar uma partida com jogadores nulos. Também é validado que o primeiro jogador tem o símbolo X e que o segundo jogador tem o símbolo O.

Este tratamento de exceções contribui para tornar a aplicação mais robusta e segura.

8. Testes Unitários

Foram implementados testes unitários com JUnit 5 para validar o comportamento das classes principais do modelo de domínio. A interface gráfica não foi testada, uma vez que os testes unitários foram focados na lógica do jogo.

Os testes foram colocados na seguinte estrutura:

```
src/test/java/tictactoe/model
├── BoardTest.java
└── GameTest.java
```

8.1. Testes da classe Board

A classe BoardTest testa os principais comportamentos do tabuleiro.

Foram considerados os seguintes cenários:

- criação de um tabuleiro vazio;
- verificação de que todas as células começam livres;
- marcação de uma célula com um símbolo válido;
- impedimento de marcação numa célula já ocupada;
- impedimento de utilização de `Symbol.EMPTY`;
- impedimento de utilização de símbolo nulo;
- validação de posições fora dos limites do tabuleiro;
- deteção de vitória numa linha;
- deteção de vitória numa coluna;
- deteção de vitória na diagonal principal;
- deteção de vitória na diagonal secundária;
- verificação de tabuleiro cheio;
- limpeza do tabuleiro através do método `reset()`.

Estes testes validam métodos como `getCell`, `isCellFree`, `markCell`, `isFull`, `checkWinner` e `reset`.

8.2. Testes da classe Game

A classe GameTest testa os principais comportamentos da partida.

Foram considerados os seguintes cenários:

- criação de um jogo no estado NOT_STARTED;
- início da partida através do método `start()`;
- validação de jogadores nulos;
- validação de símbolos incorretos nos jogadores;
- impedimento de jogadas antes do início da partida;
- confirmação de que o jogador X realiza a primeira jogada;
- alternância correta entre jogadores;
- impedimento de jogada numa célula ocupada;
- detecção de vitória do jogador X;
- detecção de empate;
- reinício da partida após uma vitória.

Estes testes validam métodos como `start`, `playMove`, `getCurrentPlayer`, `getState`, `getWinner`, `isFinished` e `getBoard`.

A existência destes testes permite verificar automaticamente se a lógica principal do jogo continua correta após alterações no código.

9. Dificuldades Encontradas

Durante o desenvolvimento do projeto surgiram várias dificuldades, principalmente relacionadas com a implementação da interface gráfica e de alguns componentes mais avançados da aplicação.

Uma das maiores dificuldades foi a criação do fundo animado da aplicação. O objetivo era tornar a interface mais dinâmica e visualmente apelativa através de elementos gráficos em movimento contínuo. Para isso, foi necessário desenvolver uma solução que permitisse animar várias partículas em simultâneo, controlando a sua posição, velocidade e direção. Além da implementação da animação, foi importante garantir que o desempenho da aplicação se mantinha estável, evitando consumos excessivos de recursos ou quebras de fluidez durante a execução.

Outra dificuldade esteve relacionada com a sincronização entre a interface gráfica e o estado interno do jogo. Sempre que um jogador realiza uma jogada, a aplicação tem de atualizar corretamente o tabuleiro, verificar condições de vitória ou empate e refletir imediatamente essas alterações na interface. Foi necessário garantir que todas estas operações ocorriam na ordem correta para evitar inconsistências visuais ou lógicas.

Também surgiram desafios na implementação da navegação entre os diferentes ecrãs da aplicação. A transição entre o menu principal, a configuração dos jogadores, o jogo e o

ecrã de resultado exigiu uma estrutura organizada para manter o código limpo e facilitar a comunicação entre as diferentes vistas. Esta dificuldade foi resolvida através da utilização da classe `AppController`, responsável por coordenar toda a navegação da aplicação.

Outra componente que exigiu atenção foi a validação das jogadas e a gestão dos estados da partida. Foi necessário garantir que não era possível jogar numa célula ocupada, realizar jogadas após o fim da partida ou iniciar o jogo com dados inválidos. Além disso, após cada jogada, o sistema tinha de verificar corretamente se existia um vencedor, se o tabuleiro estava cheio ou se o jogo deveria continuar.

Por fim, a criação de uma interface gráfica consistente e agradável também representou um desafio importante. Foram realizados vários ajustes relacionados com o posicionamento dos elementos, efeitos visuais, animações, cores e organização dos componentes gráficos. Estas melhorias contribuíram para proporcionar uma experiência de utilização mais intuitiva e visualmente atrativa.

10. Conclusão

O projeto permitiu desenvolver uma aplicação desktop completa em Java com JavaFX, aplicando de forma prática os principais conceitos de Programação Orientada a Objetos.

Foi implementado um jogo Tic Tac Toe funcional, com menu inicial, configuração dos jogadores, tabuleiro interativo, validação de jogadas, deteção de vitória, deteção de empate e ecrã de resultado.

A aplicação foi organizada de forma modular, separando o modelo de domínio, o controlo da aplicação e a interface gráfica. Esta separação tornou o código mais claro, mais organizado e mais fácil de testar.

Durante o desenvolvimento foram aplicados conceitos como encapsulamento, herança, polimorfismo e tratamento de exceções. A criação da classe abstrata `Player` e da classe `HumanPlayer` permitiu representar uma hierarquia de classes. A utilização do tipo `Player` na classe `Game` permitiu aplicar polimorfismo. O encapsulamento foi aplicado através da proteção dos atributos e da validação dos dados nos métodos públicos.

Foram também implementados testes unitários com JUnit para validar o comportamento das classes principais do modelo, garantindo maior confiança no funcionamento da aplicação.

Em conclusão, este projeto contribuiu para consolidar conhecimentos de Java, JavaFX, modelação orientada a objetos, organização modular do código, tratamento de exceções e testes unitários. O resultado final é uma aplicação funcional, estruturada e alinhada com os objetivos propostos para a unidade curricular.